

# claude-windows / f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb

## Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb\subagents\workflows\wf_ae8c9b0f-dd7\agent-a531341d4ae4cb0fc.jsonl`
- SHA-256: `9fedd4d648768bbfb28c9705f620dabea815150c7b6b556fe8339dc3a82a9b76`
- Source modified: `2026-06-22T16:39:51+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_ae8c9b0f-dd7`
- session_id: `f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb`
```

## Transcript

### 1. user (2026-06-22T16:35:38.410Z)

Review the UNCOMMITTED M12 change in the git worktree at C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.claude\\worktrees\\m11-flywheel (run `git -C the-worktree diff` + read the new files `backend/storage/gdrive_api.py` and `tests/test_storage_gdrive_api.py`). WHAT IT IS: a new `gdrive-api:// StorageBackend (GDriveApiStorage)` ? the headless Drive-API round-trip (read source + write reel back via the Drive API), distinct from the mount-only `gdrive:// (GDriveStorage`, which MUST stay untouched). Path-keyed: `gdrive-api://<path>` resolves each path component to a Drive file id via `files().list`. All Google libs are lazy-imported; a service is injectable for tests. NO DB migration. Wiring: `ref.py` (`_KNOWN` tuple + the path-keyed scheme branch), `backend/storage/__init__.py` (`_backend_class` + a hyphen-to-underscore settings lookup in `get_storage`), `backend/config/models.py` (`StorageConfig.backend/input_backend` Literals + a `gdrive_api` settings dict). Read for grounding: `backend/storage/gdrive_api.py`, `backend/storage/ref.py` (`parse_uri` + `StorageRef`), `backend/storage/__init__.py` (`_backend_class` + `get_storage`), `backend/storage/base.py` (the ABC), `backend/storage/gcs.py` (the DI template it mirrors), `backend/storage/gdrive.py` (the mount backend ? confirm untouched), `backend/config/models.py` (`StorageConfig`). Be adversarial; default to a finding if uncertain.

LENS = DRIVE-API CORRECTNESS. Scrutinize the backend logic:

- `_resolve_id` path-walk: correct for read (`create_dirs=False`, a missing component returns `None`) AND for puts parent (`create_dirs=True`, creates every missing folder including the last)? Any off-by-one? What if key is empty or just a slash? If a list query returns MULTIPLE same-named files (Drive allows that), taking `files[0]` is nondeterministic ? worth a documented caveat?
- `_q_escape`: does it correctly escape single-quotes + backslashes for a Drive q literal? Any injection risk via a crafted path/name?
- `fetch_to_local`: is `get_media().execute()` returning raw bytes the real `googleapiclient` behavior for `alt=media`? `overwrite` + `dst` handling + `makedirs` correct?
- `put`: is `_split` parent/name correct (`a/b/c.mp4` -> (`a/b`, `c.mp4`); `c.mp4` -> (`empty`, `c.mp4`) -> `root`)? It always `create()`s a new Drive file ? Drive allows duplicate names, so re-putting the same path makes a SECOND file. Is that a real concern that deserves a documented caveat or a delete-then-create?
- `stat`: parsing Drives RFC-3339 `modifiedTime` (the Z to +00:00 replace) ? robust? size as int? list: is the pagination loop (`nextPageToken`) correct, and the yielded `StorageRef` uri/key right?
- `signed_url`: not overridden, so the ABC raises `NotImplementedError` and callers fall back to `fetch_to_local`. Right call for Drive, or should it return `webViewLink`? (judgment ? note it.)

Return findings with file:line.

### 2. assistant (2026-06-22T16:35:40.765Z)

I'll analyze this M12 change to the `gdrive-api:// StorageBackend`. Let me start by examining the git diff and the new files.

### 3. user (2026-06-22T16:35:41.695Z)

```
backend/config/models.py          | 7 +++++--
backend/storage/__init__.py       | 10 ++++++--
```

```
backend/storage/ref.py | 7 +----  
docs/COMPUTE_DECOUPLED_SERVING/README.md | 3 +-  
4 files changed, 19 insertions(+), 8 deletions(-)
```

#### 4. assistant (2026-06-22T16:35:43.794Z)

Now let me read the new files and examine the git diff in detail.

#### 5. user (2026-06-22T16:35:44.957Z)

```
1      """Google Drive **API** backend (OAuth / service-account) ? ``gdrive-api://`` (M12, D14  
Northstar).  
2  
3      The cloud-native Drive round-trip: read a source from + write the stitched reel **back  
to** a user's  
4      Google Drive **via the Drive API** ? no local mount required (unlike the ``gdrive://``  
backend, which  
5      is plain local-FS over a *mounted* Drive for Desktop and stays untouched). This is the  
headless path a  
6      Cloud Run / GPU box uses.  
7  
8      ``gdrive-api://<path>`` names a path **relative to a root folder** (default the Drive  
``root``, or  
9      ``storage.gdrive-api.root_folder_id``); the backend resolves each path component to a  
Drive file id via  
10     ``files().list``. The Google libraries (``google-api-python-client`` + ``google-auth``)  
are imported  
11     **lazily** ? ``import backend.storage`` never pulls them ? and a ``service`` may be  
**injected** for  
12     fully-offline tests (mirroring ``GCSStorage(client=...)``).  
13  
14     ? **Credentials are never in config.** The real service is built from Application  
Default Credentials  
15     (a service account / workload identity) over the ``drive`` scope. **Per-user interactive  
OAuth (the  
16     consumer "connect your Drive" flow: client id/secret + refresh tokens) is OWNER-gated**  
? this backend  
17     ships the service-account/ADC build + the injected fake; the live per-user OAuth app is  
a follow-up.  
18     """  
19     from __future__ import annotations  
20  
21     import io  
22     import os  
23     from datetime import datetime  
24     from typing import Any, Iterator, Optional  
25  
26     from backend.storage.base import StorageBackend  
27     from backend.storage.ref import StorageRef, StorageStat  
28  
29     _FOLDER_MIME = "application/vnd.google-apps.folder"  
30     _DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"  
31  
32  
33     def _q_escape(name: str) -> str:  
34         """Escape a value for a Drive ``q`` string literal (single-quoted): ``\\`` then  
35         return name.replace("\\", "\\").replace("'", "\\'  
36  
37  
38     class GDriveApiStorage(StorageBackend):  
39         """Drive-API-backed storage. Construct with an injected ``service`` (tests) or let  
it build one  
40         from ADC (production). ``root_folder_id`` anchors relative ``gdrive-api://``  
paths."""
```

```

41
42     def __init__(self, *, service: Any = None, root_folder_id: str = "root", **_ignored)
-> None:
43         self._root_id = root_folder_id or "root"
44         if service is not None:
45             self._service = service
46             return
47         self._service = self._build_service()
48
49     # -- construction (real path; lazy Google imports)
-----
50     def _build_service(self) -> Any:
51         try:
52             import google.auth # type: ignore
53             from googleapiclient.discovery import build # type: ignore
54             except ImportError as e: # pragma: no cover - exercised only without the extra
installed
55                 raise RuntimeError(
56                     "the gdrive-api backend needs the 'storage-gdrive-api' extra "
57                     "(google-api-python-client + google-auth). Install it on a host that
uses "
58                     "gdrive-api:// URIs; credentials come from ADC (a service account /
workload "
59                     "identity over the Drive scope) ? never from config."
60                 ) from e
61         creds, _ = google.auth.default(scopes=[_DRIVE_SCOPE])
62         return build("drive", "v3", credentials=creds, cache_discovery=False)
63
64     def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:
65         """The upload media object for ``files().create`` ? lazy ``MediaFileUpload``
(real path).
66         A test seam: monkeypatch this to avoid the ``googleapiclient`` dependency."""
67         from googleapiclient.http import MediaFileUpload # type: ignore # pragma: no
cover
68
69         return MediaFileUpload(local_path, mimetype=content_type, resumable=True) #
pragma: no cover
70
71     # -- path -> id resolution
-----
72     def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:
73         """Walk ``key``'s path components from the root folder, returning the leaf's
file id (or
74         ``None`` if a component is missing). With ``create_dirs`` EVERY missing
component is created as
75         a folder ? ``put`` uses this on the parent-directory path (where all components
are folders),
76         so a write into a fresh subfolder works. The read paths leave it ``False`` (a
missing component
77         ? ``None``), never creating anything."""
78         parent = self._root_id
79         parts = [p for p in key.split("/") if p]
80         for part in parts:
81             q = (f"name = '{_q_escape(part)}' and '{_q_escape(parent)}' in parents "
82                 f"and trashed = false")
83             found = self._service.files().list(
84                 q=q, spaces="drive", fields="files(id,
mimeType)").execute().get("files", [])
85             if found:
86                 parent = found[0]["id"]
87             elif create_dirs:
88                 parent = self._service.files().create(
89                     body={"name": part, "mimeType": _FOLDER_MIME, "parents": [parent]},
90                     fields="id").execute()["id"]
91         else:

```

```

92         return None
93     return parent
94
95     def _split(self, key: str) -> tuple[str, str]:
96         """Split a ``gdrive-api://`` key into (parent dir path, leaf name)."""
97         clean = key.strip("/")
98         head, _, name = clean.rpartition("/")
99         return head, name
100
101     # -- StorageBackend interface
102     -----
103     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
104         if dst is None:
105             raise ValueError("GDriveApiStorage.fetch_to_local requires a dst path (no
106 local mount)")
107         dst = str(dst)
108         if os.path.exists(dst) and not overwrite:
109             return dst
110         file_id = self._resolve_id(ref.key)
111         if not file_id:
112             raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
113         data = self._service.files().get_media(fileId=file_id).execute() # alt=media ?
114         raw bytes
115         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
116         with io.open(dst, "wb") as fh:
117             fh.write(data)
118         return dst
119
120     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
121         parent_path, name = self._split(ref.key)
122         parent_id = self._resolve_id(parent_path, create_dirs=True) if parent_path else
123 self._root_id
124         if parent_id is None:
125             raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
126         self._service.files().create(
127             body={"name": name, "parents": [parent_id]},
128             media_body=self._media_body(str(local_path), content_type),
129             fields="id").execute()
130         return ref
131
132     def exists(self, ref) -> bool:
133         return self._resolve_id(ref.key) is not None
134
135     def stat(self, ref) -> StorageStat:
136         file_id = self._resolve_id(ref.key)
137         if not file_id:
138             raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
139         meta = self._service.files().get(
140             fileId=file_id, fields="size, modifiedTime, md5Checksum,
141 mimeType").execute()
142         modified = None
143         if meta.get("modifiedTime"):
144             # Drive returns RFC-3339 (e.g. "2026-06-22T10:00:00.000Z"); normalize the
145 trailing Z.
146             modified = datetime.fromisoformat(meta["modifiedTime"].replace("Z",
147 "+00:00"))
148         return StorageStat(
149             size=int(meta.get("size") or 0), modified=modified,
150             etag=meta.get("md5Checksum"), content_type=meta.get("mimeType"))
151
152     def list(self, prefix) -> Iterator[StorageRef]:
153         folder_id = self._resolve_id(prefix.key) if prefix.key.strip("/") else
154 self._root_id
155         if not folder_id:
156             return

```

```

149         base = prefix.key.strip("/")
150         page_token: Optional[str] = None
151         while True:
152             resp = self._service.files().list(
153                 q=f'"{_q_escape(folder_id)}" in parents and trashed = false',
154                 spaces="drive", fields="nextPageToken, files(id, name)",
155                 pageToken=page_token).execute()
156             for f in resp.get("files", []):
157                 key = f"{base}/{f['name']}" if base else f["name"]
158                 yield StorageRef("gdrive-api", "", key, f"gdrive-api://{key}")
159             page_token = resp.get("nextPageToken")
160             if not page_token:
161                 break
162
163     def delete(self, ref) -> None:
164         file_id = self._resolve_id(ref.key)
165         if file_id:
166             self._service.files().delete(fileId=file_id).execute()
167

```

## 6. assistant (2026-06-22T16:35:46.885Z)

Now let me read the test file:

## 7. user (2026-06-22T16:35:47.891Z)

```

1      """M12 ? the ``gdrive-api://`` OAuth/API StorageBackend (D14 Northstar).
2
3      Fully offline: a tiny in-memory ``_FakeDrive`` stands in for the googleapiclient Drive
v3 ``service``
4      (injected, mirroring the GCS fake-client tests). No ``google-api-python-client`` /
network needed.
5      Asserts: scheme parsing, registry wiring, the hyphen?underscore settings lookup, and the
full
6      path-keyed round-trip (put ? exists ? stat ? fetch ? list ? delete).
7      """
8      import re
9
10     import pytest
11
12     from backend.config import AppConfig
13     from backend.storage import get_storage
14     from backend.storage.gdrive_api import GDriveApiStorage
15     from backend.storage.ref import parse_uri
16
17
18     # ----- the in-memory fake
Drive service
19
20     class _Exec:
21         def __init__(self, value):
22             self._value = value
23
24         def execute(self):
25             return self._value
26
27
28     class _FakeFiles:
29         def __init__(self, drive):
30             self.d = drive
31
32         def list(self, q="", spaces=None, fields=None, pageToken=None):
33             name_m = re.search(r"name = '([^\']*)'", q)
34             parent_m = re.search(r"'([^\']*)' in parents", q)
35             parent = parent_m.group(1) if parent_m else None

```

```

36         out = []
37         for nid, n in self.d.nodes.items():
38             if parent is not None and parent not in n["parents"]:
39                 continue
40             if name_m and n["name"] != name_m.group(1):
41                 continue
42             if nid == "root":
43                 continue
44             out.append({"id": nid, "name": n["name"], "mimeType": n["mimeType"]})
45         return _Exec({"files": out}) # one page, no nextPageToken
46
47     def create(self, body=None, media_body=None, fields=None):
48         nid = self.d.new_id()
49         self.d.nodes[nid] = {
50             "name": body["name"], "mimeType": body.get("mimeType", "application/octet-
stream"),
51             "parents": list(body.get("parents", [])),
52         }
53         if media_body is not None: # in tests _media_body is patched to return the
local path
54             with open(media_body, "rb") as fh:
55                 self.d.content[nid] = fh.read()
56         return _Exec({"id": nid})
57
58     def get_media(self, fileId=None):
59         return _Exec(self.d.content.get(fileId, b""))
60
61     def get(self, fileId=None, fields=None):
62         n = self.d.nodes[fileId]
63         data = self.d.content.get(fileId, b"")
64         return _Exec({"size": str(len(data)), "modifiedTime":
"2026-06-22T10:00:00.000Z",
65                       "md5Checksum": "deadbeef", "mimeType": n["mimeType"]})
66
67     def delete(self, fileId=None):
68         self.d.nodes.pop(fileId, None)
69         self.d.content.pop(fileId, None)
70         return _Exec({})
71
72
73     class _FakeDrive:
74         def __init__(self):
75             self.nodes = {"root": {"name": "root", "mimeType": "folder", "parents": []}}
76             self.content = {}
77             self._n = 0
78
79         def new_id(self):
80             self._n += 1
81             return f"id{self._n}"
82
83         def files(self):
84             return _FakeFiles(self)
85
86
87     @pytest.fixture
88     def be(monkeypatch):
89         """A GDriveApiStorage on a fake service, with _media_body patched to the local path
(no
90         googleapiclient dependency ? the fake `create` reads the bytes from that path)."""
91         monkeypatch.setattr(GDriveApiStorage, "_media_body", lambda self, p, ct: p)
92         return GDriveApiStorage(service=_FakeDrive())
93
94
95     # ----- scheme +
registry wiring

```

```

96
97     def test_parse_uri_gdrive_api_is_path_keyed():
98         ref = parse_uri("gdrive-api://Coaching/2026/clip.mp4")
99         assert (ref.backend, ref.bucket, ref.key) == ("gdrive-api", "",
"Coaching/2026/clip.mp4")
100         assert ref.uri == "gdrive-api://Coaching/2026/clip.mp4"
101
102
103     def test_unknown_scheme_still_rejected():
104         with pytest.raises(ValueError):
105             parse_uri("gdrive-zzz://x")
106
107
108     def test_config_accepts_gdrive_api_backend():
109         cfg = AppConfig.model_validate({"storage": {"backend": "gdrive-api",
"input_backend": "gdrive-api"}})
110         assert cfg.storage.backend == "gdrive-api"
111         assert cfg.storage.input_backend == "gdrive-api"
112
113
114     def test_get_storage_wires_gdrive_api_and_reads_underscore_settings(monkeypatch):
115         """Registry resolves 'gdrive-api' ? GDriveApiStorage, and its settings come from the
116         `gdrive_api` field (hyphen?underscore lookup) merged with overrides (here, the
injected service)."""
117         monkeypatch.setattr(GDriveApiStorage, "_media_body", lambda self, p, ct: p)
118         cfg = {"backend": "gdrive-api", "gdrive_api": {"root_folder_id": "FOLDER42"}}
119         be = get_storage("gdrive-api", config=cfg, service=_FakeDrive())
120         assert isinstance(be, GDriveApiStorage)
121         assert be._root_id == "FOLDER42"
122
123
124     # ----- full
round-trip
125
126     def test_roundtrip_put_exists_stat_fetch_list_delete(be, tmp_path):
127         src = tmp_path / "clip.mp4"
128         src.write_bytes(b"VIDEO-BYTES")
129         ref = parse_uri("gdrive-api://Coaching/2026/clip.mp4")
130
131         be.put(str(src), ref)                # creates the Coaching/ + 2026/ folders
+ the file
132         assert be.exists(ref)
133         assert be.stat(ref).size == len(b"VIDEO-BYTES")
134         assert be.stat(ref).etag == "deadbeef"
135
136         dst = tmp_path / "out" / "clip.mp4"
137         be.fetch_to_local(ref, str(dst))
138         assert dst.read_bytes() == b"VIDEO-BYTES"
139
140         listed = [r.uri for r in be.list(parse_uri("gdrive-api://Coaching/2026"))]
141         assert listed == ["gdrive-api://Coaching/2026/clip.mp4"]
142
143         be.delete(ref)
144         assert not be.exists(ref)
145
146
147     def test_fetch_missing_raises(be, tmp_path):
148         with pytest.raises(FileNotFoundError):
149             be.fetch_to_local(parse_uri("gdrive-api://nope/missing.mp4"), str(tmp_path /
"x"))
150
151
152     def test_fetch_requires_dst(be):
153         with pytest.raises(ValueError):
154             be.fetch_to_local(parse_uri("gdrive-api://a/b.mp4"), None)

```

```

155
156
157     def test_put_at_root_uses_root_folder(be, tmp_path):
158         src = tmp_path / "top.json"
159         src.write_text("{}")
160         ref = parse_uri("gdrive-api://top.json")
161         be.put(str(src), ref)
162         assert be.exists(ref)
163
164
165     def test_build_service_without_libs_raises_actionable_error(monkeypatch):
166         """No injected service + the Google libs absent ? a clear, owner-actionable error
(not ImportError
167         leaking through). googleapiclient is not installed in the test env, so this
exercises the real path."""
168         with pytest.raises(RuntimeError, match="storage-gdrive-api"):
169             GDriveApiStorage()
170

```

## 8. assistant (2026-06-22T16:35:50.842Z)

Now let me check the git diff for the wiring changes and examine the related files:

## 9. user (2026-06-22T16:35:52.422Z)

```

diff --git a/backend/config/models.py b/backend/config/models.py
index 9c8cf77..015c6a0 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -315,7 +315,7 @@ class StorageConfig(BaseModel):
     Google Drive (`gdrive.mount_root`). Per-backend settings are loose dicts passed to the
    constructor ?
    **secrets are never stored here**, only endpoints / regions / file paths / source-selectors
    (boto3/google auth chains supply credentials). See `backend/storage/`.
-    backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
+    backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
    # Output root for produced artifacts. "" => fall back to output.output_dir (non-breaking).
    output_root: str = ""
    # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
@@ -326,7 +326,7 @@ class StorageConfig(BaseModel):
    # (`"gcs"` + "bucket/x.mp4" => `gcs://bucket/x.mp4`). An explicit scheme on the input
    # (`gs://`/`s3://`) or an absolute local path ALWAYS wins. Non-local inputs are
    fetched to a
    # scratch dir before processing (see `backend.storage.acquire_local_input`).
-    input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
+    input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
    input_root: str = ""
    # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
    # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
@@ -343,6 +343,9 @@ class StorageConfig(BaseModel):
    s3: dict[str, Any] = Field(default_factory=dict)
    gcs: dict[str, Any] = Field(default_factory=dict)
    gdrive: dict[str, Any] = Field(default_factory=dict)
+    # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the hyphen?underscore
    lookup in
+    # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive scope.
+    gdrive_api: dict[str, Any] = Field(default_factory=dict)


```

```

class DeploymentConfig(BaseModel):
diff --git a/backend/storage/__init__.py b/backend/storage/__init__.py
index 6aaf278..fbd2206 100644
--- a/backend/storage/__init__.py

```

```
+++ b/backend/storage/__init__.py
@@ -88,7 +88,10 @@ def _backend_class(name: str):
    if name == "gdrive":
        from backend.storage.gdrive import GDriveStorage
        return GDriveStorage
-    raise ValueError(f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive)")
+    if name == "gdrive-api":
+        from backend.storage.gdrive_api import GDriveApiStorage
+        return GDriveApiStorage
+    raise ValueError(f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-
api)")
```

```
def get_storage(backend: Optional[str] = None, *,
@@ -98,7 +101,10 @@ def get_storage(backend: Optional[str] = None, *,
    ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
    cfg = config or {}
    name = backend or cfg.get("backend", "local")
-    settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
+    # Per-backend settings live under the backend name; tolerate the hyphen?underscore form so
+    a
+    # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a hyphen
+    # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
+    settings: Dict[str, Any] = dict(cfg.get(name) or cfg.get(name.replace("-", "_")) or {})
    settings.update(overrides)
    return _backend_class(name)(**settings)
```

```
diff --git a/backend/storage/ref.py b/backend/storage/ref.py
```

```
index 60e2430..f40a63a 100644
```

```
--- a/backend/storage/ref.py
```

```
+++ b/backend/storage/ref.py
```

```
@@ -19,7 +19,7 @@ from datetime import datetime
    from typing import Optional
```

```
_SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
-_KNOWN = ("local", "s3", "gcs", "gdrive")
+_KNOWN = ("local", "s3", "gcs", "gdrive", "gdrive-api")
```

```
@dataclass(frozen=True)
@@ -59,8 +59,9 @@ def parse_uri(uri: str) -> StorageRef:
    rest = s[m.end():]
    if scheme == "gs":
        scheme = "gcs"
-    if scheme in ("local", "gdrive"):
-        # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My Drive".
+    if scheme in ("local", "gdrive", "gdrive-api"):
+        # Path-keyed, no bucket: local FS, a path under the mounted Drive, or a path under a
+    Drive-API
+    # root folder (gdrive-api resolves the path ? a Drive file id).
    return StorageRef(scheme, "", rest, s)
    if scheme in _KNOWN:
        bucket, _, key = rest.partition("/")
```

```
diff --git a/docs/COMPUTE_DECOUPLED_SERVING/README.md b/docs/COMPUTE_DECOUPLED_SERVING/README.md
```

```
index 56e3084..e60d644 100644
```

```
--- a/docs/COMPUTE_DECOUPLED_SERVING/README.md
```

```
+++ b/docs/COMPUTE_DECOUPLED_SERVING/README.md
```

```
@@ -124,7 +124,7 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per
| M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate
wiring | M7 | ? owner (privacy/counsel) | ? | [#290](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; **+
exposure-safety boot posture** in `startup.py` ? edge-auth-on requires the `allowed_video_root`
jail + CF team/AUD or the server refuses to start, see changelog; consent cols + ToS/DPA ?
owner/counsel) |
```

```
| M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner
approval | ? | ? |
| **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload +
Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval**
owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden
TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model
climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). |
M7,M9 | ? owner (consent/privacy) | ? | **skeleton landed** (see changelog):
`backend/flywheel.py` + `FlywheelConfig` (default-OFF) + gated `_maybe_run_flywheel` hook,
**consent faked-deny** ; live activation ? owner/counsel (consent schema) + owned-model/golden
data |
-| **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source
from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user
OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount
backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
+| **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source
from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user
OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount
backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | **backend
landed** (see changelog): `backend/storage/gdrive_api.py` `GDriveApiStorage` + `gdrive-api://`
scheme + ADC build; hermetic fake-Drive tests. **Live per-user OAuth app ? owner** |
```

```
**Testing strategy is a first-class deliverable** ?
[`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ?
[`runlogs/`](runlogs/).
```

```
@@ -169,3 +169,4 @@ Legend: ? Todo . ? In progress . ? Done . ? Gated. **One small PR per
- 2026-06-21 ? **M5 code landed** ([#286](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/286)): `backend/config/startup.py` ? per-profile fail/warn-fast startup checks
(cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches =
warnings, the runner healthcheck stays the authority), wired into the server
(`_enforce_startup_or_exit` ? exit 1) + the worker (`cmd_infer`/`cmd_train` ? rc 2), default-OFF
(probe gated behind an active profile ? the worker stays torch-free at `profile=""`). **L4
profile-parity test** (`tests/test_profile_parity.py`): same stub trajectory ? byte-identical
windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process.
Adversarial 4-lens review folded (`wf_34671ac1`). Suite green, ruff+mypy clean. **? M5 ? ? the
real-GPU truly-local runlog (L5) is owner-side** (template pre-staged in `runlogs/`).
- 2026-06-21 ? **M9 hardening: exposure-safety boot posture** (alpha-shareable "tunnel-to-a-
box" enabler): extended `backend/config/startup.py` with a **server-only exposure posture** (new
`include_exposure=True` from `_enforce_startup_or_exit`; the worker stays unaffected ? it reads
`--video-uri`, not untrusted tenant paths). Gated on `security.edge_auth_enabled` (**default-OFF
? strict no-op**, byte-identical to today): when the owner flips edge auth ON to expose the
engine behind the CF-Access gate, three checks fire at **boot** ? `EXPOSED_NO_VIDEO_ROOT`
(**fatal**: `allowed_video_root` jail unset ? a tenant could read an arbitrary local file),
`EXPOSED_EDGE_AUTH_INCOMPLETE` (**fatal**: CF team-domain/AUD missing ? lifts the current first-
request 401 to boot), `EXPOSED_AUTH_EXTRA_MISSING` (**warn**: `PyJWT[crypto]` not importable ?
would 500 per request). So a half-configured exposure fails fast instead of leaking/500-ing. +15
unit tests (default-OFF no-op, worker-ignores, both-fatals-compose-with-profile-checks, server-
exit, WARN-only-no-exit). Adversarial 3-lens review (`wf_35b735a9`) confirmed default-OFF byte-
identical + no bypass; folded a **whitespace-drift** (aligned `auth.resolve_tenant` to
`.strip()`) team/AUD so "configured" means the same at boot + at request time) + named the config
fields in the edge-auth-incomplete message. Pairs with the appliance **T3** network posture (raw
`:8000` refused ? owner-run). Suite green, ruff+mypy clean. **? backstops M9 ? the actual
exposure flip + CF team/AUD/ingress + key rotation stay owner-side. (Deferred to the PR-2
runbook: document that `upload_dir` must sit under `allowed_video_root`, and that you MUST set
`edge_auth_enabled=True` when exposing ? these checks only fire then.)**
- 2026-06-22 ? **M11 skeleton landed** (data-flywheel harness, D12): `backend/flywheel.py`
(`capture_job` ? per-video `VideoWorkspace` via `backend.storage.put`; `shadow_eval` ?
`experiment.compare`/`compare_unlabeled`; `propose_promotion` ?
`promotion.promote_if_served_safe` with Gemini=control/owned=proposal; `run_for_job`
orchestrator) + `FlywheelConfig` (**default-OFF**) + a gated `_maybe_run_flywheel` job-
completion hook mirroring M8's cost hook. **TWO independent guards keep it INERT in
production:** `flywheel.enabled=False` (default) AND `consent_attested` is a **FAKED hard-deny**
(returns False for every job) ? so even enabled it captures **nothing** until the real consent
schema lands. This is the PLUMBING only; it reuses the real workspace/eval/promotion/storage
```

code so it works the day consent + the owned model + golden labels are wired. **\*\*NO DB migration\*\*** (the skeleton persists no consent). +13 tests (default-OFF no-op, enabled-but-consent-denied no-op, consented-capture writes the manifest, delegation of shadow-eval/promote, the hook is best-effort). Suite green, ruff+mypy clean. **\*\*? M11 ? ? LIVE activation is owner/counsel** (the consent ToS/DPA + schema ? the consent columns) + owner/GPU (the owned model + golden corpus for shadow-eval/promotion). ? the future M9-consent migration is now `v9` (v7 = `locale`, v8 = the compute-picker `compute` column ? the schema ladder drifts as siblings land; check `PRAGMA user\_version` before authoring).**\*\***

+ 2026-06-22 ? **\*\*M12 backend landed\*\*** (`gdrive-api://` OAuth/API StorageBackend, D14 Northstar): `backend/storage/gdrive\_api.py` `GDriveApiStorage(StorageBackend)` ? the headless **\*\*Drive-API\*\*** round-trip (read source + write reel back), distinct from the mount-only `gdrive://` (left untouched). Path-keyed `gdrive-api://<path>` resolves each component ? a Drive file id via `files().list`; implements `fetch\_to\_local` (`get\_media().execute()` bytes), `put` (folder-creating `files().create` + a `\_media\_body` seam), `exists`/`stat`/`list`/`delete`. Wired the scheme into `ref.py` (`\_KNOWN` + the path-keyed branch), `\_\_init\_\_.\_backend\_class`, and the `StorageConfig.backend`/`input\_backend` Literals + a `gdrive\_api` settings dict (read via a **\*\*hyphen+underscore\*\*** lookup in `get\_storage`). **\*\*All Google libs lazy-imported\*\*** (`import backend.storage` never pulls them); a `service` is **\*\*injectable\*\*** ? fully-offline tests via an in-memory fake Drive (round-trip + scheme + registry + the underscore-settings lookup); `\_build\_service` (ADC over the Drive scope) raises a clear `storage-gdrive-api`-extra error when libs/creds are absent. **\*\*NO DB migration.\*\*** Adversarial 2-lens review (`wf\_ae8c9b0f`). Suite green (1466), ruff+mypy clean. **\*\*? M12 ? ? the LIVE per-user interactive OAuth app** (consumer "connect your Drive": client id/secret + refresh tokens) is owner-gated; the backend ships the service-account/ADC build + the fake. Reads no secrets from config (creds via ADC only).**\*\***

## 10. assistant (2026-06-22T16:35:54.368Z)

Now let me examine the base storage backend and compare with existing backends:

## 11. user (2026-06-22T16:35:55.468Z)

```

1      """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3      Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4      live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5      pulls ``boto3`` / ``google-cloud-storage`` (`gdrive` is pure local-FS over a mounted
Drive ?
6      no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7      """
8      from __future__ import annotations
9
10     import os
11     from abc import ABC, abstractmethod
12     from typing import Iterator
13
14     from backend.storage.ref import StorageRef, StorageStat
15
16
17     class StorageBackend(ABC):
18         """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19         worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21         @abstractmethod
22         def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                             *, overwrite: bool = False) -> str:
24             """Make the object available as a local file; return its local path. For the
25             ``local`` backend this is an identity passthrough (the object is already
local)."""
26
27         @abstractmethod
28         def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
29                 content_type: "str | None" = None) -> StorageRef:

```

```

30         """Upload/copy a local file to ``ref``; return ``ref``."""
31
32     @abstractmethod
33     def exists(self, ref: StorageRef) -> bool: ...
34
35     @abstractmethod
36     def stat(self, ref: StorageRef) -> StorageStat: ...
37
38     @abstractmethod
39     def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
40         """Yield a ``StorageRef`` per object under ``prefix``."""
41
42     @abstractmethod
43     def delete(self, ref: StorageRef) -> None: ...
44
45     def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
46         """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
47         sign (``local`` cannot; Drive only emulates) ? callers must treat
48         ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
49         raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
50

```

## 12. assistant (2026-06-22T16:35:57.372Z)

Now let me check the gdrive.py and gcs.py backends to understand the expected patterns:

## 13. user (2026-06-22T16:35:57.994Z)

```

1     """Google Drive backend ? reads Drive files through a **locally mounted** Google Drive.
2
3     We make a deliberate, baked-in assumption: **to use Google Drive files you mount Google
Drive
4     for Desktop**, which exposes Drive as an ordinary filesystem path. That's a fair
requirement ? it
5     needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on
access. So
6     ``gdrive://`` is just a *portable, OS-agnostic alias* over that mount: it names a path
**relative to
7     your "My Drive" folder**, and everything else is plain local-filesystem I/O (the same
code the
8     ``local`` backend uses).
9
10         gdrive://Sharing/clip.mp4    ->    <mount>/My Drive/Sharing/clip.mp4
11
12     Why an alias instead of just a raw path? Portability: the mount root differs per machine
13     (Windows ``G:\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My
Drive``), so the same
14     ``gdrive://`` URI resolves on any box with Drive mounted. The root is **auto-detected**,
or set it
15     explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a
secret).
16
17     A headless box (a GPU VM, CI) has **no** mount ? there, stage the file to a bucket and
use
18     ``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path. This backend
intentionally does NOT
19     talk to the Drive API. When the mount (or a file in it) is missing, errors point the
user at the
20     fix: install Drive for Desktop and include the folder in syncing so it can be pulled
locally.
21     """
22     from __future__ import annotations
23
24     import glob

```

```

25     import os
26     import string
27     from typing import Iterator, Optional
28
29     from backend.storage.base import StorageBackend
30     from backend.storage.local import LocalStorage
31     from backend.storage.ref import StorageRef, StorageStat
32
33     _INSTALL_URL = "https://www.google.com/drive/download/"
34
35     # Shared remedy line ? how to make a folder available locally. Drive for Desktop streams
files,
36     # but a folder only appears once it's part of your synced Drive (and a *shared* folder
must first
37     # be added to "My Drive"). The same fix resolves both "not mounted" and "mounted but
missing".
38     _SYNC_REMEDY = (
39         "make sure the folder is INCLUDED in syncing so its files can be pulled locally: in
Google "
40         "Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open
Drive in the "
41         "browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'.
Drive then "
42         "downloads the bytes on first access."
43     )
44
45
46     def _not_mounted_msg(tried: Optional[str]) -> str:
47         return (
48             "Google Drive is not mounted, so the gdrive:// backend cannot resolve any
path.\n"
49             f"  Remedy:\n"
50             f"  1. Install Google Drive for Desktop and sign in: {_INSTALL_URL}\n"
51             f"  2. Then {_SYNC_REMEDY}\n"
52             f"  3. If your 'My Drive' is not at the default location (Windows G:\\My Drive,
macOS "
53             f"~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or "
54             f"storage.gdrive.mount_root to point at it (auto-detect tried: {tried!r})."
55         )
56
57
58     def _not_synced_msg(root: str, rel: str, uri: str) -> str:
59         return (
60             f"Google Drive is mounted ({root!r}) but {uri!r} is not present locally "
61             f"(expected at {os.path.join(root, *rel.split('/')!r}).\n"
62             f"  This usually means the folder is not synced yet ? {_SYNC_REMEDY}\n"
63             f"  (Install/repair Drive for Desktop if needed: {_INSTALL_URL})"
64         )
65
66
67     def _autodetect_mount_root() -> Optional[str]:
68         """Find a mounted Google Drive 'My Drive' folder, or ``None``. Covers the Google
Drive for
69         Desktop defaults: macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`` and
Windows
70         ``<letter>:\\My Drive`` (``G:`` is the install default). Linux/other ? configure
explicitly."""
71         home = os.path.expanduser("~")
72         for p in sorted(glob.glob(os.path.join(home, "Library", "CloudStorage",
"GoogleDrive*", "My Drive"))):
73             if os.path.isdir(p):
74                 return p
75         if os.name == "nt":
76             # G is the Drive-for-Desktop default; then scan the rest of the assignable
letters.

```

```

77         for letter in ["G"] + [c for c in string.ascii_uppercase if c != "G"]:
78             p = f"{letter}:\\My Drive"
79             if os.path.isdir(p):
80                 return p
81     return None
82
83
84     class GDriveStorage(StorageBackend):
85         def __init__(self, *, mount_root: Optional[str] = None, **_ignored) -> None:
86             root = mount_root or os.environ.get("GDRIVE_MOUNT_ROOT") or
87             _autodetect_mount_root()
88             if not root or not os.path.isdir(root):
89                 raise RuntimeError(_not_mounted_msg(root))
90             self._root = os.path.abspath(root)
91             self._local = LocalStorage(base_dir=self._root) # all I/O is plain local-FS
92         under the mount
93         def _local_ref(self, ref: StorageRef) -> StorageRef:
94             """Translate a ``gdrive://<rel>`` ref into a ``local`` ref keyed by an OS path
95             relative to
96             the mount root (``LocalStorage`` joins it under ``base_dir``)."""
97             rel = ref.key.lstrip("/")
98             key = os.path.join(*rel.split("/")) if rel else ""
99             return StorageRef("local", "", key, f"local://{key}")
100
101         def _require_present(self, ref: StorageRef) -> None:
102             """Guard the read paths: a path missing from the mount almost always means the
103             folder
104             isn't synced ? raise with that remedy instead of a cryptic FileNotFoundError
105             downstream."""
106             rel = ref.key.lstrip("/")
107             abs_path = os.path.join(self._root, *rel.split("/")) if rel else self._root
108             if not os.path.exists(abs_path):
109                 raise FileNotFoundError(_not_synced_msg(self._root, rel, ref.uri))
110
111         def _to_gdrive_uri(self, abs_path: str) -> str:
112             """The inverse: an absolute path under the mount ? its ``gdrive://`` URI."""
113             rel = os.path.relpath(abs_path, self._root).replace(os.sep, "/")
114             return f"gdrive://{rel}"
115
116         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
117             # The file is already local (via the mount): no dst => identity passthrough to
118             the mount
119             # path; a dst => copy out (a byte-stable scratch copy), exactly as the local
120             backend does.
121             self._require_present(ref)
122             return self._local.fetch_to_local(self._local_ref(ref), dst,
123             overwrite=overwrite)
124
125         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
126             self._local.put(local_path, self._local_ref(ref), content_type=content_type)
127             return ref
128
129         def exists(self, ref) -> bool:
130             return self._local.exists(self._local_ref(ref))
131
132         def stat(self, ref) -> StorageStat:
133             self._require_present(ref)
134             return self._local.stat(self._local_ref(ref))
135
136         def list(self, prefix) -> Iterator[StorageRef]:
137             for r in self._local.list(self._local_ref(prefix)):
138                 uri = self._to_gdrive_uri(r.key)
139                 yield StorageRef("gdrive", "", uri[len("gdrive://"):], uri)

```

```

134         def delete(self, ref) -> None:
135             self._local.delete(self._local_ref(ref))
136

```

#### 14. assistant (2026-06-22T16:36:00.309Z)

Now let me check the gcs.py backend to understand the DI pattern better:

#### 15. user (2026-06-22T16:36:01.312Z)

```

1     """Google Cloud Storage backend ? first-class object buckets.
2
3     Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4     install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5     (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6     never from config. A ``client`` may be injected for tests.
7     """
8     from __future__ import annotations
9
10    from datetime import timedelta
11    from typing import Any, Iterator, Optional
12
13    from backend.storage.base import StorageBackend
14    from backend.storage.ref import StorageRef, StorageStat
15
16
17    class GCSStorage(StorageBackend):
18        def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                    signed_url_ttl: int = 3600, **_ignored) -> None:
20            self._ttl = signed_url_ttl
21            if client is not None:
22                self._client = client
23            return
24            from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25            self._client = storage.Client(project=project)
26
27        def _blob(self, ref: StorageRef):
28            return self._client.bucket(ref.bucket).blob(ref.key)
29
30        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31            if dst is None:
32                raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33            import os
34            dst = str(dst)
35            if os.path.exists(dst) and not overwrite:
36                return dst
37            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38            self._blob(ref).download_to_filename(dst)
39            return dst
40
41        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42            self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43            return ref
44
45        def exists(self, ref) -> bool:
46            return bool(self._blob(ref).exists())
47
48        def stat(self, ref) -> StorageStat:
49            b = self._blob(ref)
50            b.reload()
51            return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,

```

```

52             content_type=b.content_type)
53
54     def list(self, prefix) -> Iterator[StorageRef]:
55         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56             yield StorageRef("gcs", prefix.bucket, blob.name,
57 f"gcs://{prefix.bucket}/{blob.name}")
58
59     def delete(self, ref) -> None:
60         self._blob(ref).delete()
61
62     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
63         return self._blob(ref).generate_signed_url(
64             version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)

```

## 16. assistant (2026-06-22T16:36:03.863Z)

Now let me check the ref.py file to see the context around StorageRef and StorageStat:

## 17. user (2026-06-22T16:36:04.278Z)

```

1     """Storage value objects + URI parsing for the pluggable storage layer.
2
3     A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5         <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
6 /data/x.mp4)
7         local://<path>             -> local    (local:///abs or local://rel/p)
8         s3://<bucket>/<key>        -> s3       (AWS + R2 + DO Spaces + MinIO via
9 endpoint_url)
10        gcs://<bucket>/<key>       -> gcs      (gs:// accepted, normalized to gcs://)
11        gdrive://<path>            -> gdrive   (path under your locally-MOUNTED Google
12 Drive "My Drive")
13
14     No cloud SDKs are imported here ? this module is pure stdlib so ``import
15 backend.storage``
16     stays cheap.
17     """
18     from __future__ import annotations
19
20     import re
21     from dataclasses import dataclass
22     from datetime import datetime
23     from typing import Optional
24
25     _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
26     _KNOWN = ("local", "s3", "gcs", "gdrive", "gdrive-api")
27
28     @dataclass(frozen=True)
29     class StorageStat:
30         """Lightweight object metadata (uniform across backends)."""
31         size: int
32         modified: Optional[datetime] = None
33         etag: Optional[str] = None
34         content_type: Optional[str] = None
35
36     @dataclass(frozen=True)
37     class StorageRef:
38         """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
39 ``gdrive`` it is
40         empty and ``key`` carries the path (a filesystem path, or a path under the mounted
41 Drive)."""
42         backend: str

```

```

39     bucket: str
40     key: str
41     uri: str
42
43     @property
44     def name(self) -> str:
45         return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48     def parse_uri(uri: str) -> StorageRef:
49         """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51         A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52         have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53         """
54         s = str(uri)
55         m = _SCHEME_RE.match(s)
56         if not m:
57             return StorageRef("local", "", s, f"local://{s}")
58         scheme = m.group(1).lower()
59         rest = s[m.end():]
60         if scheme == "gs":
61             scheme = "gcs"
62         if scheme in ("local", "gdrive", "gdrive-api"):
63             # Path-keyed, no bucket: local FS, a path under the mounted Drive, or a path
under a Drive-API
64             # root folder (gdrive-api resolves the path ? a Drive file id).
65             return StorageRef(scheme, "", rest, s)
66         if scheme in _KNOWN:
67             bucket, _, key = rest.partition("/")
68             return StorageRef(scheme, bucket, key, s)
69         raise ValueError(f"unsupported storage URI scheme: {scheme}://{s} (in {s!r})")
70
71
72     def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
73         """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
74
75         Precedence (first match wins):
76         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
77         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
78         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
79         (``gs://bucket/videos`` + ``x.mp4`` ? ``gs://bucket/videos/x.mp4``).
80         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``gcs`` +
``bucket/x`` ?
81         ``gcs://bucket/x``).
82         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
83
84         So the default (``input_backend="local"`` , ``input_root=""``) is an identity
passthrough for every
85         path used today, preserving byte-for-byte behaviour.
86         """
87         s = str(raw)
88         if _SCHEME_RE.match(s):           # 1. explicit scheme wins
89             return s
90         # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
91         #     os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
92         #     "/data/x" is NOT absolute). Anchored == a leading "/" or "\\ ", or a Windows
drive prefix "X:".

```

```

93         if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
94             return s
95         if input_root:                                # 3. join a relative name under the configured
root
96             return input_root.rstrip("/") + "/" + s.lstrip("/")
97         if input_backend and input_backend != "local": # 4. prefix a bucket/key with the
declared backend
98             return f"{input_backend}://{s}"
99         return s                                     # 5. bare/relative local path ? unchanged
100

```

## 18. assistant (2026-06-22T16:36:06.364Z)

Now let me check the backend/storage/\_\_init\_\_.py file more thoroughly:

## 19. user (2026-06-22T16:36:07.484Z)

```

1      """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3      URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4      lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5      ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6      today's behaviour byte-for-byte; ``gdrive`` likewise needs none ? it reads a locally
MOUNTED
7      Google Drive (Google Drive for Desktop) as plain files.
8
9      Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10     ``list_uris`` / ``get_storage``; ``local`` (and bare paths) make ``fetch`` an
identity
11     passthrough. See ``docs/DECOUPLED_COMPUTE``.
12     """
13     from __future__ import annotations
14
15     import os
16     import tempfile
17     from typing import Any, Dict, Iterator, Optional
18
19     from backend.storage.base import StorageBackend
20     from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22     __all__ = [
23         "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24         "get_storage", "fetch", "put", "list_uris",
25         "acquire_local_input", "input_is_local", "resolve_input_ref",
26     ]
27
28
29     def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30         """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31         if storage_config is None:
32             return {}
33         if hasattr(storage_config, "model_dump"):
34             return storage_config.model_dump() # type: ignore[no-any-return]
35         return dict(storage_config)
36
37
38     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
40         config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
41         callers at the API/CLI boundary catch that and return a clean client error (never a
500).

```

```

42
43     Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44     ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45     sc = _storage_dict(storage_config)
46     uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                             input_root=sc.get("input_root", ""))
48     return parse_uri(uri)
49
50
51     def input_is_local(raw: str, storage_config: Any = None) -> bool:
52         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55         ``resolve_input_ref``)."""
56         return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59     def acquire_local_input(raw: str, storage_config: Any = None, *,
60                             scratch_parent: "str | None" = None) -> str:
61         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66         path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67         regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68         worker's ``cmd_infer`` already ships."""
69         sc = _storage_dict(storage_config)
70         ref = resolve_input_ref(raw, sc)
71         if ref.backend == "local":
72             return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
73         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage
91         if name == "gdrive-api":
92             from backend.storage.gdrive_api import GDriveApiStorage
93             return GDriveApiStorage
94         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive|gdrive-api)")

```

```

95
96
97     def get_storage(backend: Optional[str] = None, *,
98                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
99         """Construct a backend. ``backend`` (or ``config['backend']`` , default 'local')
selects
100         the implementation; its per-backend settings come from ``config[backend]`` merged
with
101         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
102         cfg = config or {}
103         name = backend or cfg.get("backend", "local")
104         # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
105         # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
106         # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
107         settings: Dict[str, Any] = dict(cfg.get(name) or cfg.get(name.replace("-", "_")) or
{})
108         settings.update(overrides)
109         return _backend_class(name)(**settings)
110
111
112     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
113              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
114         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
115         (no copy); for s3/gcs it downloads to ``dst``."""
116         ref = parse_uri(uri)
117         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
118
119
120     def put(local_path: "str | os.PathLike[str]", uri: str, *,
121            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
122         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
123         ref = parse_uri(uri)
124         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
125         return uri
126
127
128     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
129         """Yield the URI of every object under ``prefix_uri``."""
130         ref = parse_uri(prefix_uri)
131         for r in get_storage(ref.backend, config=config).list(ref):
132             yield r.uri
133

```

## 20. assistant (2026-06-22T16:36:09.684Z)

Now let me check the backend/config/models.py to see the full context:

## 21. user (2026-06-22T16:36:11.114Z)

```

310
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
314         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to

```

```

the constructor ?
316      **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317      (boto3/google auth chains supply credentials). See ``backend/storage/``.
318      backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
319      # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320      output_root: str = ""
321      # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322      # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323      # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324      # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
325      # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326      # (``"gcs"`` + "bucket/x.mp4" => ``gs://bucket/x.mp4``). An explicit scheme on the
input
327      # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328      # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329      input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
330      input_root: str = ""
331      # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332      # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
333      # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334      # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335      # the legacy flat layout is used unchanged.
336      single_folder_per_video: bool = False
337      # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338      workspace_root: str = ""
339      # Cloud namespace prefix under the root so per-video folders sit tidily beside
340      # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341      workspace_prefix: str = "workspaces"
342      local: dict[str, Any] = Field(default_factory=dict)
343      s3: dict[str, Any] = Field(default_factory=dict)
344      gcs: dict[str, Any] = Field(default_factory=dict)
345      gdrive: dict[str, Any] = Field(default_factory=dict)
346      # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
347      # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
348      gdrive_api: dict[str, Any] = Field(default_factory=dict)
349
350
351      class DeploymentConfig(BaseModel):
352          """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
353          knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
354
355          **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
356          behaves exactly as before this section existed. A profile is opt-in (``config.json``
357          ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
358          *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
359          these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs

```

## 22. assistant (2026-06-22T16:36:14.740Z)

Perfect. Now let me carefully analyze the code for the correctness issues. Let me trace through critical paths:

## 23. user (2026-06-22T16:36:17.028Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\m11-flywheel
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 9 items

tests/test_storage_gdrive_api.py::test_parse_uri_gdrive_api_is_path_keyed PASSED [ 11%]
tests/test_storage_gdrive_api.py::test_unknown_scheme_still_rejected PASSED [ 22%]
tests/test_storage_gdrive_api.py::test_config_accepts_gdrive_api_backend PASSED [ 33%]
tests/test_storage_gdrive_api.py::test_get_storage_wires_gdrive_api_and_reads_underscore_setting
s PASSED [ 44%]
tests/test_storage_gdrive_api.py::test_roundtrip_put_exists_stat_fetch_list_delete PASSED [ 55%]
tests/test_storage_gdrive_api.py::test_fetch_missing_raises PASSED [ 66%]
tests/test_storage_gdrive_api.py::test_fetch_requires_dst PASSED [ 77%]
tests/test_storage_gdrive_api.py::test_put_at_root_uses_root_folder PASSED [ 88%]
tests/test_storage_gdrive_api.py::test_build_service_without_libs_raises_actionable_error PASSED
[100%]

===== 9 passed in 0.29s =====
```

## 24. assistant (2026-06-22T16:36:19.489Z)

Good. Now let me trace through the `_resolve_id` logic and potential edge cases, especially around path parsing and empty key handling:

## 25. user (2026-06-22T16:36:29.693Z)

```
C:\Users\avidu\AppData\Local\Temp\test_gdrive_edge_cases.py:54: SyntaxWarning: invalid escape
sequence '\s'
    ("back\\slash", "backslash"),
_split tests:
root file          'clip.mp4'          => ('', 'clip.mp4')
nested            'Coaching/2026/clip.mp4'    => ('Coaching/2026', 'clip.mp4')
leading slash     '/clip.mp4'          => ('', 'clip.mp4')
multiple leading slashes '///clip.mp4'          => ('', 'clip.mp4')
leading slash nested '/Coaching/2026/clip.mp4'    => ('Coaching/2026', 'clip.mp4')
trailing slash   'Coaching/2026/'      => ('Coaching', '2026')
both slashes     '/Coaching/2026/'      => ('Coaching', '2026')
empty            ''                  => ('', '')
just slash       '/'                  => ('', '')
multiple slashes '///'                => ('', '')

_resolve_id path walk:
key='clip.mp4', create_dirs=False
parts: ['clip.mp4']
step 0: looking for 'clip.mp4' in parent='root'
final parent: id_0
key='Coaching/2026/clip.mp4', create_dirs=False
parts: ['Coaching', '2026', 'clip.mp4']
step 0: looking for 'Coaching' in parent='root'
step 1: looking for '2026' in parent='id_0'
step 2: looking for 'clip.mp4' in parent='id_1'
final parent: id_2
key='', create_dirs=False
```

```

    parts: []
    final parent: root
key='/', create_dirs=False
    parts: []
    final parent: root

```

Edge case: multiple files with same name

If list() returns multiple files with the same name, taking files[0] is nondeterministic

This is a Drive API behavior: names are NOT unique, only (name, parent) is unique

\_q\_escape tests:

|                      |                |                    |
|----------------------|----------------|--------------------|
| normal               | 'normal'       | => 'normal'        |
| file with apostrophe | "file's.mp4"   | => "file's.mp4"    |
| backslash            | 'back\\slash'  | => 'back\\\\slash' |
| both slash and quote | "both'\x08oth" | => "both'\x08oth"  |

## 26. assistant (2026-06-22T16:36:31.935Z)

Now let me look at one critical issue more closely - the put() behavior with duplicate names:

## 27. user (2026-06-22T16:36:40.673Z)

Drive API duplicate-name behavior:

1. put('a/x.mp4') -> creates 'a/x.mp4' (id\_001)
2. put('a/x.mp4') -> creates ANOTHER 'a/x.mp4' (id\_002)
3. list('a/') -> returns both files with same name
4. \_resolve\_id('a/x.mp4') -> finds TWO files, returns files[0] (oldest)  
This is nondeterministic! If the API changes sort order, reads break.
5. files().delete(id\_002) -> deletes only id\_002, leaves id\_001

So re-putting is idempotent in behavior but NOT in storage (orphaned files accumulate)

The fix would be:

- Check if the file exists via \_resolve\_id()
  - If exists, delete it FIRST: files().delete(existing\_id)
  - Then create the new one
- OR
- Accept that re-puts create orphans (document as a caveat)

## 28. assistant (2026-06-22T16:36:42.673Z)

Now let me check the actual list behavior in the fake test to understand it better:

## 29. user (2026-06-22T16:36:44.347Z)

```

32     def list(self, q="", spaces=None, fields=None, pageToken=None):
33         name_m = re.search(r"name = '([^\']*)'", q)
34         parent_m = re.search(r"'([^\']*)' in parents", q)
35         parent = parent_m.group(1) if parent_m else None
36         out = []
37         for nid, n in self.d.nodes.items():
38             if parent is not None and parent not in n["parents"]:
39                 continue
40             if name_m and n["name"] != name_m.group(1):
41                 continue
42             if nid == "root":
43                 continue
44             out.append({"id": nid, "name": n["name"], "mimeType": n["mimeType"]})
45         return _Exec({"files": out}) # one page, no nextPageToken
46
47     def create(self, body=None, media_body=None, fields=None):
48         nid = self.d.new_id()
49         self.d.nodes[nid] = {
50             "name": body["name"], "mimeType": body.get("mimeType", "application/octet-

```

```
stream"),
51             "parents": list(body.get("parents", [])),
```

### 30. assistant (2026-06-22T16:36:46.376Z)

Now let me test an actual re-put scenario to see if it creates duplicates:

### 31. user (2026-06-22T16:36:53.247Z)

After first put:

```
exists: True
Fake service nodes: {'root': {'name': 'root', 'mimeType': 'folder', 'parents': []}, 'id1':
{'name': 'a', 'mimeType': 'application/vnd.google-apps.folder', 'parents': ['root']}, 'id2':
{'name': 'b.mp4', 'mimeType': 'application/octet-stream', 'parents': ['id1']}
```

After second put:

```
exists: True
Fake service nodes: {'root': {'name': 'root', 'mimeType': 'folder', 'parents': []}, 'id1':
{'name': 'a', 'mimeType': 'application/vnd.google-apps.folder', 'parents': ['root']}, 'id2':
{'name': 'b.mp4', 'mimeType': 'application/octet-stream', 'parents': ['id1']}, 'id3': {'name':
'b.mp4', 'mimeType': 'application/octet-stream', 'parents': ['id1']}
```

```
list('a/'): ['gdrive-api://a/b.mp4', 'gdrive-api://a/b.mp4']
```

### 32. assistant (2026-06-22T16:36:55.496Z)

Excellent! I found a real issue. Now let me check the stat modifiedTime parsing more carefully:

### 33. user (2026-06-22T16:37:01.012Z)

```
2026-06-22T10:00:00.000Z      => 2026-06-22 10:00:00+00:00 ?
2026-06-22T10:00:00Z        => 2026-06-22 10:00:00+00:00 ?
2026-06-22T10:00:00+00:00   => 2026-06-22 10:00:00+00:00 ?
```

Edge case: missing modifiedTime  
modified = None

Edge case: size field  
int(meta.get('size') or 0) = 0  
int(meta.get('size') or 0) = 0

### 34. assistant (2026-06-22T16:37:02.972Z)

Good. Now let me check the actual Drive API documentation behavior for get\_media():

### 35. user (2026-06-22T16:37:09.580Z)

fetch\_to\_local behavior:

1. Line 111: data = self.\_service.files().get\_media(fileId=file\_id).execute()
  - This assumes all files in Drive can be downloaded as raw bytes
  - Google Docs (sheets, slides) CANNOT be downloaded directly
  - For those, you need to export them (files().export\_media(fileId=..., mimeType=...))

Is this a caveat to document?

Yes - if a user uploads Google Sheets/Slides, fetch\_to\_local will fail

The code should either:

- A) Document this as a limitation
- B) Detect Google Docs mimeType and raise a clear error
- C) Support exporting (more complex)

### 36. assistant (2026-06-22T16:37:11.549Z)

Now let me check for the signed\_url judgment:

### 37. user (2026-06-22T16:37:19.486Z)

Drive signed\_url options:

#### 1. webViewLink:

- Returns a view-only link in Drive
- Does NOT bypass sharing (requires user to have Drive access)
- LLM can't click it; browser gets a login prompt

#### 2. webContentLink:

- Drive's internal download redirect
- Still requires Drive auth if shared restricted
- LLM can't use it meaningfully

#### 3. Not implemented (current):

- Callers fall back to fetch\_to\_local
- Bytes flow through the app (safe, slow, suitable for processing)

JUDGMENT: Not implementing it is CORRECT.

Why: Drive doesn't have true anonymous signed URLs like S3.

Drive's sharing model requires the user to have access.

fetch\_to\_local fallback is the right semantic for Drive.

### 38. assistant (2026-06-22T16:37:21.526Z)

Now let me check the exact line in the put() function with potential issues:

### 39. user (2026-06-22T16:37:23.049Z)

```
117         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
118             parent_path, name = self._split(ref.key)
119             parent_id = self._resolve_id(parent_path, create_dirs=True) if parent_path else
self._root_id
120             if parent_id is None:
121                 raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
122             self._service.files().create(
123                 body={"name": name, "parents": [parent_id]},
124                 media_body=self._media_body(str(local_path), content_type),
125                 fields="id").execute()
126             return ref
```

### 40. assistant (2026-06-22T16:37:25.070Z)

Now let me test the \_split behavior more deeply with edge cases:

### 41. user (2026-06-22T16:37:32.965Z)

\_split behavior:

|                              |                  |                                   |
|------------------------------|------------------|-----------------------------------|
| directory reference          | 'Coaching/2026/' | => parent='Coaching', name='2026' |
| directory path without slash | 'Coaching/2026'  | => parent='Coaching', name='2026' |
| single dir with slash        | '2026/'          | => parent='', name='2026'         |
| single dir without slash     | '2026'           | => parent='', name='2026'         |

Problem: 'Coaching/2026/' is treated as putting a FILE named '2026'

Instead of a directory. The user probably wanted to list the directory.

This is detected at the call site (put vs list), not here.

put() with 'Coaching/2026/':

1. \_split('Coaching/2026/') => ('Coaching', '2026')
2. Creates parent 'Coaching' folder
3. Creates file '2026' (no extension!) as a CHILD of 'Coaching'
4. Not as a folder, but as a file with bytes

This is a USER error (should call put with a FILE, not a directory),  
but the error message won't be clear.

#### 42. assistant (2026-06-22T16:37:36.098Z)

Now let me verify the query escape logic more carefully with injection tests:

#### 43. user (2026-06-22T16:37:45.335Z)

```
C:\Users\avidu\AppData\Local\Temp\test_q_escape.py:2: SyntaxWarning: invalid escape sequence '\`'
    """Escape a value for a Drive ``q`` string literal (single-quoted): ``\`` then ``'``,"""
C:\Users\avidu\AppData\Local\Temp\test_q_escape.py:12: SyntaxWarning: invalid escape sequence '\p'
    ("c:\path\file", "windows path"),
Query injection tests:
  normal file
    input: 'normal'
    query: name = 'normal' and 'parent' in parents and trashed = false

  single quote
    input: "file'mp4"
    query: name = 'file'mp4' and 'parent' in parents and trashed = false

  query injection attempt 1
    input: "a' and trashed = false"
    query: name = 'a' and trashed = false' and 'parent' in parents and trashed = false

  query injection attempt 2 - tautology
    input: "a' or 'b' = 'b"
    query: name = 'a' or 'b' = 'b' and 'parent' in parents and trashed = false

  already escaped quote
    input: "a'"
    query: name = 'a'' and 'parent' in parents and trashed = false

  windows path
    input: 'c:\\path\x0cile'
    query: name = 'c:\\path
ile' and 'parent' in parents and trashed = false
```

Security analysis:

1. The escape order is CORRECT: \ first, then ''  
This ensures backslash doesn't escape the following escape
2. Injection test: "a' and trashed = false"  
Escaped: "a' and trashed = false"  
Query: name = 'a' and trashed = false' and 'parent' in parents and trashed = false  
Result: The injected ' is escaped to ', so it's a LITERAL quote in the name search  
NOT an injection!
3. Injection test: "a\' (trailing backslash then quote)"  
Input: "a\\'"  
Escaped: "a\\\\'"  
Query: name = 'a\\'' and 'parent' in parents and trashed = false  
Result: Safe - the \ is escaped to \\, and the ' to '

#### 44. assistant (2026-06-22T16:37:47.138Z)

Let me trace through the actual escape more carefully:

#### 45. user (2026-06-22T16:38:05.001Z)

```
<stdin>:62: SyntaxWarning: invalid escape sequence '\`'
Input:  "a' and trashed = false"
Escaped: "a' and trashed = false"
Query:  "name = 'a' and trashed = false' and 'parent' in parents and trashed = false"
```

Query (plain): name = 'a' and trashed = false' and 'parent' in parents and trashed = false

Drive API parsing of the query:

The q parser sees: name = 'a' and trashed = false'...  
It reads the quote-delimited string as: a' and trashed = false  
Wait, that's WRONG! Let me re-check...

Corrected analysis:

Python: "a' and trashed = false"  
UTF-8 bytes: b"a' and trashed = false"  
Sent to Drive: a' and trashed = false

Drive q-string parsing:

Sees: name = 'a' and trashed = false' and...

Step-by-step Drive q-string parsing of: name = 'a' and trashed = false' and...

Position 0-7: name = '  
Position 8-9: a\  
Position 10: ' <- this is an ESCAPED quote, so it's a literal ' character  
Position 11-32: (literally) ' and trashed = false'  
NO WAIT. The closing ' for the string isn't until after 'false'

Let me trace the actual escaped string:

"a' and trashed = false"  
In the q query: name = 'a' and trashed = false'  
With actual chars: name = 'a\' and trashed = false'

Drive q parser (per Google Sheets Functions docs):

Inside '...', \ escapes the next character  
So a' is: a followed by (escaped quote) = literal a'  
The query becomes: name = 'a' and trashed = false' and 'parent'...  
But that's TWO quoted strings: 'a' THEN 'and trashed = false'  
That's a SYNTAX ERROR in the q language!

Actually, I need to check the ACTUAL Drive q syntax...

According to Google: Drive query syntax uses single quotes for string literals  
Inside a single-quoted string, use \ for backslash and ' for quote

So \_q\_escape does: .replace('\', '\\').replace("'", "\'")  
Which Python shows as: .replace(' ', '\').replace("'", "\'")

Test: 'hello'world' (literal string with a quote in the middle)

Input: "hello'world"  
Escaped: "hello'world"  
In query: name = 'hello'world'  
Drive sees: name = 'hello'world'  
Which parses as: hello'world (literal string)  
CORRECT!

#### 46. assistant (2026-06-22T16:38:07.015Z)

The output is confusing due to terminal escaping. Let me check the actual Drive API documentation approach more carefully:

#### 47. user (2026-06-22T16:38:21.755Z)

<stdin>:49: SyntaxWarning: invalid escape sequence '\ '  
<stdin>:50: SyntaxWarning: invalid escape sequence '\ '  
Test string with quote in middle:  
Original: hello'world (8 chars: h,e,l,l,o,',w,o,r,l,d)  
After escape: 11 chars  
0: 'h'  
1: 'e'  
2: 'l'  
3: 'l'  
4: 'o'

```
5: ''
6: 'w'
7: 'o'
8: 'r'
9: 'l'
10: 'd'
```

The escape produces: hello'world

So the query is: name = 'hello'world'

Drive parses it as: name = (literal string) with content: hello'world

? CORRECT - the backslash-quote is an escape sequence meaning a literal quote

Query injection test:

Original: a' and trashed = false

Escaped: a' and trashed = false

Full query: name = 'a' and trashed = false' and 'parent' in parents and trashed = false

When Drive parses this, it sees:

name = 'a' and trashed = false' and ...

The first quote opens, content is: a' and trashed = false

The string ends at the second ', then 'and' is parsed as a boolean operator

This is WRONG - it breaks the q syntax!

AHA! There IS an injection vulnerability!

Actually, let's re-examine. The `_q_escape` output for "a' and":

Input: a'

Output: a'

Byte sequence: ['a', '"']

'a'""

So `_q_escape('a')` produces: a\ ' (three characters)

In the q query: name = 'a\ '...'

The backslash escapes the next character (the backslash itself)

No wait, that's only if there are TWO backslashes

Let me read the code again:

`.replace('\', '\\')` - Python strings show this as:

`replace(backslash, two-backslashes)`

`.replace("'", '"')`

`replace(quote, backslash-quote)`

So for 'a'b':

Step 1: replace backslashes - 'a\b' becomes 'a\\b' (4 backslashes)

Step 2: replace quotes - no quotes in 'a\\b'

Result: a\b (with 4 backslashes? No, that doesn't make sense...)

Actual execution:

original = 'a\x08'

after `.replace(backslash, double-backslash)` = 'a\x08'

original = "a'b"

after `.replace(quote, backslash-quote)` = "a'b"

#### 48. assistant (2026-06-22T16:38:23.845Z)

The terminal escaping is confusing things. Let me write a clearer test:

#### 49. user (2026-06-22T16:38:32.754Z)

<stdin>:44: SyntaxWarning: invalid escape sequence '\d'

<stdin>:44: SyntaxWarning: invalid escape sequence '\d'

Test 1: String with apostrophe

Input length: 4

```
Input bytes: b"it's"
Output length: 4
Output bytes: b"it's"
Expected: it's (with backslash-apostrophe)
```

Test 2: Query injection attempt

```
Input: file' and trashed=false
Escaped: file' and trashed=false
Escaped bytes: b"file' and trashed=false"
Query: name = 'file' and trashed=false' and 'parent' in parents
Query bytes: b"name = 'file' and trashed=false' and 'parent' in parents"
```

Drive query escaping rules (from Google):

Inside 'string', use \ for literal backslash, and ' for literal apostrophe

So our escape function SHOULD produce:

```
.replace(backslash, double-backslash)
.replace(apostrophe, backslash-apostrophe)
```

Let's check the actual byte outputs:

```
"it's"          -> "it's"          (bytes: b"it's")
'a\x08'         -> 'a\x08'         (bytes: b'a\x08')
"c\\d'e"        -> "c\\d'e"        (bytes: b"c\\d'e")
```

**50. assistant (2026-06-22T16:38:35.043Z)**

I see - the output is showing that the escape isn't being applied. Let me check the source code again - there might be an issue:

**51. user (2026-06-22T16:38:36.832Z)**

```
33     def _q_escape(name: str) -> str:
34         """Escape a value for a Drive `q` string literal (single-quoted): `` then
35         ``"""
36         return name.replace("\\", "\\").replace("'", "\'")
```

**52. assistant (2026-06-22T16:38:39.354Z)**

The source code looks right. The issue with my test output is Python terminal escaping. Let me verify the actual behavior works correctly:

**53. user (2026-06-22T16:38:43.603Z)**

Exit code 1

```
Test 1: "hello'world"
Escaped: "hello'world"
Actual: hello'world
Length: 11
```

```
Test 2: "file'and"
Escaped: "file'and"
Query: name = 'file'and'
```

cat: /tmp/escape\_test.txt: No such file or directory

**54. assistant (2026-06-22T16:38:46.166Z)**

Let me verify the escape function works by examining it directly in Python:

**55. user (2026-06-22T16:38:50.460Z)**

Testing \_q\_escape function:

Input 1: hello'world  
Expected: hello\'world (with backslash-quote)  
Got: hello\'world  
repr: "hello\'world"

Input 2: file.mp4  
Got: file.mp4

In the actual Drive q query:  
name = 'hello\'world' and parent\_id in parents

#### 56. assistant (2026-06-22T16:38:54.328Z)

Perfect! The escape IS working correctly. The repr showed `"hello\'world"` which is the correct escape. Now I have all the information I need. Let me compile my findings:

#### 57. user (2026-06-22T16:39:34.160Z)

Structured output provided successfully

#### 58. assistant (2026-06-22T16:39:51.124Z)

Perfect. Let me now provide a summary of my findings to ensure the user has the context:

### Summary

I've completed a thorough 2-lens adversarial review of the M12 gdrive-api:// StorageBackend change. Here are the key findings:

#### MAJOR ISSUE: put() Creates Duplicate Files on Re-puts

**\*\*Location:\*\*** ``backend/storage/gdrive_api.py:117-126``

The ``put()`` method always calls ``files().create()`` without checking if the file already exists. Since Drive allows duplicate names under the same parent, re-putting the same path creates a second file instead of overwriting it.

**\*\*Evidence:\*\*** I tested this scenario:

- First ``put('gdrive-api://a/b.mp4')`` creates file ``id2`` named ``b.mp4``
- Second ``put('gdrive-api://a/b.mp4')`` creates file ``id3`` named ``b.mp4``
- Subsequent reads return ``files[0]`` (oldest), leaving ``id3`` orphaned
- ``list()`` shows both duplicates

**\*\*Fix:\*\*** Before creating, check if the file exists via ``_resolve_id()``. If yes, delete it first via ``files().delete(existing_id)``, then create the new one. This makes `put()` properly idempotent.

The test suite doesn't catch this because ``test_roundtrip_put_exists_stat_fetch_list_delete`` only puts once per path.

#### MINOR ISSUE: fetch\_to\_local() Fails on Google Docs

**\*\*Location:\*\*** ``backend/storage/gdrive_api.py:102-115``

``fetch_to_local()`` uses ``files().get_media().execute()`` which works for regular files (videos, PDFs) but fails for Google Docs (Sheets, Slides, Presentations). These MIME types require ``files().export_media()`` instead.

**\*\*Fix:\*\*** Document the limitation in the docstring, or detect Google-Docs MIME types and raise a friendly error: `"gdrive-api: {uri} is a Google Docs spreadsheet/presentation. Export it to PDF/XLSX in Drive first."`

#### MINOR CAVEAT: Nondeterministic Result When Duplicate Names Exist

**Location:** `backend/storage/gdrive\_api.py:86`

If Drive contains multiple files with the same name under a parent (allowed by Drive API), `\_resolve\_id()` returns `files[0]`, which depends on Drive's internal sort order (not guaranteed stable).

**Fix:** Document this edge case in the class docstring. The fake test never creates duplicates, so it doesn't expose this.

#### **CORRECT IMPLEMENTATIONS:**

- ? **Query escaping** (`\_q\_escape`): Correctly escapes backslash then quote?no injection risk
- ? **DateTime parsing** (`stat`): RFC-3339 Z?+00:00 replacement is robust; handles missing fields gracefully
- ? **Pagination** (`list`): Correctly loops `nextPageToken` until final page
- ? **signed\_url judgment**: Correctly **not** overridden?Drive has no true anonymous signed URLs; `fetch_to_local` fallback is the right semantic
- ? **Wiring**: Hyphen?underscore lookup in `get\_storage` is clever and correct; Pydantic can't have hyphens in field names
- ? **Lazy imports & DI**: All Google libs lazy-imported; `service` injectable for tests; no SDK pull at import time
- ? **No DB migration**: Correct?just a new backend wiring

The backend is well-designed overall. The duplicate-file issue is the main correctness concern for production use.